Project Chimera — Day 1 Report

Role: Forward Deployed Engineer
Mission: Architect a factory that builds autonomous influencers
Candidate: Aman Kassahun Wassie
Date: June 20, 2026
Repository: https://github.com/amaneth/project-chimera

---

1. Executive Summary

Day 1 focused on understanding the autonomous agent landscape, translating the Project Chimera SRS into architecture decisions, and establishing a specification-first repository foundation.

My core conclusion is that Project Chimera should not be treated as a content scheduling app. It should be designed as governed agentic infrastructure: a system where autonomous influencer agents can perceive trends, plan campaigns, generate content, interact publicly, remember context, and eventually participate in governed economic actions.

The architecture direction I selected is a **Hierarchical Swarm** using the **Planner → Worker → Judge** pattern. This pattern separates strategy, execution, and governance. The Planner decomposes campaign goals into tasks, Workers execute atomic tasks using approved tools, and Judges review outputs for quality, policy, safety, budget, and state consistency before any public or financial action is committed.

The repository was structured around Spec-Driven Development. The goal is to make `specs/` the source of truth so that future AI coding agents can enter the codebase, understand the system boundaries, and build features without relying on vague prompts or hidden assumptions.

---

2. Research Summary

2.1 The Trillion Dollar AI Software Development Stack

The key insight from the AI software development stack reading is that AI-assisted development is moving toward a structured loop:

Plan → Code → Review

This directly supports the challenge's Spec-Driven Development requirement. AI agents are most useful when they are given clear context, explicit constraints, tests, and review loops. Without this structure, they can generate plausible but incorrect code.

For Project Chimera, this means the repository itself must act as context for future agents. The `specs/` directory, `.cursor/rules` or equivalent agent rules, tests, and CI pipeline are not secondary artifacts. They are part of the product architecture.

Key takeaways applied to Chimera:

- Specs should guide implementation before code is written.
- Tests should define expected behavior before implementation.
- AI coding agents must be instructed to read `specs/` first.
- Source control should show how decisions evolved.
- Review and validation are necessary for both human and AI-generated work.

2.2 OpenClaw and the Agent Social Network

OpenClaw and Moltbook show a future where AI agents are not only assistants responding to humans, but participants in agent-facing social spaces. Agents can post, comment, share knowledge, join topic forums, use skills, and communicate with other agents.

Project Chimera fits naturally into this direction. Chimera agents are public-facing autonomous influencers, so they may eventually need to interact with both humans and other agents. This makes Chimera a potential participant in an Agent Social Network.

However, the research also highlights major risks. Agent-to-agent messages cannot be trusted blindly. External agents may send unsafe instructions, prompt injection attempts, misleading content, or requests outside the Chimera agent's allowed scope.

Key takeaways applied to Chimera:

- Agent-to-agent communication must be structured and auditable.
- External agent messages should be treated as untrusted input.
- Collaboration requests should pass through Judge review.
- Sensitive or financial requests should escalate to human review.
- Chimera needs clear protocols for identity, capability, availability, trust, refusal, and escalation.

2.3 Moltbook: Social Media for Bots

Moltbook demonstrates the idea of a bot-native social network where agents can create posts, comment, and share automation knowledge. The important lesson is not only that agents can communicate, but that they need repeatable social protocols to do so safely.

For Chimera, this suggests that public-facing influencer agents should not simply respond to messages as raw text. They need structured communication formats that preserve intent, source, risk, permissions, and auditability.

Key protocols Chimera may need:

- Identity and disclosure protocol
- Capability advertisement protocol
- Availability and status protocol
- Collaboration request protocol
- Trust and reputation protocol
- Content exchange protocol
- Approval, refusal, and escalation protocol

This directly informed the bonus `specs/openclaw_integration.md` specification.

2.4 Project Chimera SRS

The SRS defines Project Chimera as an Autonomous Influencer Network, not a basic automation tool. The system must support persistent agents with persona, memory, planning, creative generation, social action, governance, and potentially economic agency.

The most important architectural pattern from the SRS is the **FastRender-style Planner → Worker → Judge swarm**:

- **Planner:** turns high-level campaign goals into concrete tasks.
- **Worker:** executes atomic tasks using approved tools and skills.
- **Judge:** validates results, checks safety, applies confidence thresholds, prevents stale commits, and escalates to humans when needed.

The SRS also makes MCP a key architectural boundary. External systems such as social platforms, news sources, media generation tools, memory systems, and commerce services should be accessed through MCP tools and resources, not direct calls from agent core logic.

Key SRS requirements applied to my architecture:

- Planner → Worker → Judge as the core execution loop
- MCP-first external integration
- Human-in-the-loop review for risky, sensitive, low-confidence, or financial actions
- SOUL.md persona definition for each agent
- PostgreSQL for canonical transactional data
- Redis for queues, short-term state, and budget/rate counters
- Weaviate for long-term semantic memory
- Object storage for generated media
- Java 21 records for immutable DTOs
- Java 21 virtual threads for scalable worker execution
- Optimistic Concurrency Control using `state_version`

- Audit logs for planning, execution, review, and approval decisions

---

3. Architectural Approach

3.1 Chosen Agent Pattern: Hierarchical Swarm

I selected a **Hierarchical Swarm** using the **Planner → Worker → Judge** pattern.

A simple sequential chain is easier to understand, but it does not fit Chimera's needs. Chimera must support many simultaneous campaigns, trend reactions, comment replies, content drafts, media tasks, and review decisions. A sequential chain would create bottlenecks and make governance harder to isolate.

A hierarchical swarm is more appropriate because it separates responsibility:

- The Planner owns strategy and task decomposition.
- Workers own execution of isolated atomic tasks.
- The Judge owns review, safety, policy, and state consistency.

This separation makes the system easier to test, easier to audit, and safer to scale.

3.2 Human-in-the-Loop Strategy

Human review is placed at the Judge layer.

This means Workers can generate drafts and artifacts, but they do not directly publish or execute risky actions. The Judge evaluates Worker outputs and applies policy before any action is finalized.

The confidence policy is:

- `confidence_score > 0.90`: may auto-approve if not sensitive or financial
- `confidence_score 0.70–0.90`: route to asynchronous human approval
- `confidence_score < 0.70`: reject, retry, or safely abort
- Sensitive topics always escalate regardless of confidence score

Sensitive topics include:

- politics
- health advice
- financial advice
- legal claims

Financial actions require additional budget governance through a CFO Judge or equivalent review mechanism.

3.3 MCP Integration Strategy

MCP is treated as the boundary between the agent core and the outside world.

The agent core should not directly call social media APIs, news APIs, vector databases, image/video generation APIs, or commerce systems. Instead, those capabilities should be exposed as MCP tools and resources.

This gives Chimera several advantages:

- Platform-specific code is isolated outside the agent core.
- Credentials and rate limits can be managed at the MCP boundary.
- Tool usage can be logged and audited.
- Future platform changes are less likely to break the core agent architecture.
- AI coding agents have clearer integration boundaries.

Examples of future MCP tools/resources include:

- `fetch_trends`
- `search_memory`
- `post_content`
- `generate_media`
- `check_budget`
- `get_wallet_balance`

---

4. Data and Infrastructure Decisions

4.1 Database Strategy

I selected a hybrid storage strategy rather than choosing only SQL or only NoSQL.

For high-velocity video and content metadata, PostgreSQL should remain the source of truth because the system needs structure, auditability, relationships, tenant isolation, and clear governance records.

Recommended data split:

- **PostgreSQL:** tenants, agents, campaigns, tasks, worker results, judge verdicts, human reviews, audit logs, budgets, wallet references
- **Redis:** task queues, review queues, HITL queues, short-term memory, rate-limit counters, budget counters
- **Weaviate:** long-term semantic memory, persona memory, trend embeddings, similarity search

- **Object Storage:** generated images, videos, thumbnails, and intermediate media assets

This avoids forcing one database to handle every workload. PostgreSQL protects canonical records, Redis handles fast-changing operational state, Weaviate supports semantic retrieval, and object storage handles large binary assets.

4.2 Java 21 Foundation

The repository was initialized with a Java 21 Maven foundation. The purpose of this setup is not to implement the full system on Day 1, but to establish a professional enterprise-ready base for Day 2 tests and future implementation.

The Java direction is:

- Java 21+
- Maven build system
- JUnit 5 for tests
- Jackson for JSON contracts
- Immutable DTOs using Java records
- Virtual threads for future scalable Worker execution

This aligns with the challenge's emphasis on Java 21, strong typing, testability, and agent-friendly implementation boundaries.

---

5. Specification Artifacts Created

The repository contains a `specs/` directory designed to act as the source of truth for future AI-assisted implementation.

`specs/_meta.md`

Defines repository-level purpose, scope, constraints, non-goals, source-of-truth rules, governance expectations, and technical constraints.

Key decisions:

- Specs come before implementation.
- `specs/` is the authoritative design source.
- Planner → Worker → Judge is the core pattern.
- MCP is required for external integrations.
- HITL is required for sensitive, risky, low-confidence, or financial actions.
- Agent-to-agent messages are untrusted input.

`specs/functional.md`

Defines what the system must do from a behavioral perspective.

Covered areas include:

- Persona loading from SOUL.md
- Trend perception
- Campaign planning
- Task creation
- Worker execution
- Judge review
- HITL escalation
- Content publishing approval
- Memory updates
- Agent-to-agent communication
- Budget governance
- Audit trails
- Disclosure and failure handling

`specs/technical.md`

Defines concrete contracts for future implementation and tests.

Covered areas include:

- Runtime components
- Java 21 data modeling rules
- Agent API JSON contracts
- MCP tool contracts
- PostgreSQL schema
- Mermaid ERD
- Redis keys and queues
- OCC using `state_version`
- Security and governance constraints
- Testability requirements

`specs/openclaw_integration.md`

Defines how Chimera may safely participate in an OpenClaw/Moltbook-style Agent Social Network.

Covered protocols include:

- Identity and disclosure
- Availability and status
- Capability advertisement
- Collaboration requests

- Content exchange
- Trust and reputation
- Approval, refusal, and escalation

The integration is intentionally designed as a future MCP adapter/server, not direct core logic.

---

6. AI-Assisted Engineering Process

I used AI tools as engineering assistants, not as a substitute for architectural judgment.

My workflow was:

1. Read and summarize the research materials.
2. Write my own initial intent and architecture decisions.
3. Use GitHub Copilot to expand drafts based on my notes and SRS summary.
4. Review and refine the generated output manually.
5. Correct architectural inconsistencies such as ID consistency, database relationships, governance gaps, and multi-tenancy concerns.
6. Commit progress incrementally.

This process matches the challenge's emphasis on AI-assisted work. The goal was not to blindly copy AI output, but to use AI to accelerate drafting while I retained responsibility for decisions, review, and final alignment.

Examples of human decisions I made before AI expansion:

- Choosing Hierarchical Swarm over Sequential Chain
- Treating MCP as the external integration boundary
- Placing HITL at the Judge layer
- Choosing PostgreSQL + Redis + Weaviate + Object Storage as the storage strategy
- Requiring `state_version` for OCC
- Treating agent-to-agent messages as untrusted input

---

7. Risks and Open Questions

Several risks remain for future implementation:

1. **Prompt injection risk**
   Agent-to-agent and public social messages may include malicious instructions. These must be sanitized and reviewed.
2. **Sensitive topic classification**
   The system needs reliable classification for politics, health, legal, and financial content.

3. **Human review latency**
   HITL protects safety, but delayed review may reduce responsiveness during fast-moving trends.
4. **Cost and budget control**
   Media generation, LLM inference, and commerce actions require strong budget limits.
5. **Platform volatility**
   Social media APIs change frequently, so MCP adapters must absorb platform-specific changes.
6. **State consistency**
   Workers may act on stale campaign context, so Judge-level OCC using `state_version` is required.
7. **Multi-tenancy**
   Agent memory, campaign data, budgets, and media assets must remain isolated across tenants.

---

8. Day 1 Outcome

By the end of Day 1, the repository has been shaped into a specification-first foundation for Project Chimera.

Completed Day 1 outcomes:

- Deep research notes
- Architecture strategy document
- Git repository initialized
- Tenx MCP Sense connected
- Java 21 Maven foundation created
- Master specification structure created
- Meta specification drafted
- Functional specification drafted
- Technical specification drafted
- OpenClaw integration specification drafted

The result is not a quick prototype. It is a context-engineered repository intended to guide future AI agents, tests, and implementation work with minimal ambiguity.

9. Conclusion

Project Chimera should be built as governed autonomous infrastructure. The system must allow agents to move quickly, but not without review, traceability, and policy enforcement.

The architectural direction selected on Day 1 is:

- Spec-Driven Development
- Planner → Worker → Judge swarm architecture
- MCP-first integration boundary
- Human-in-the-loop governance
- Hybrid storage model
- Java 21 immutable DTOs and virtual-thread-ready execution
- Auditability and safety before autonomy
- Agent social network readiness through structured protocols

This foundation prepares the repository for Day 2, where the next focus will be context engineering, agent rules, skill definitions, failing tests, automation, CI/CD, and AI review governance.